

USER GUIDE · V1.7

The Cutscene Builder

For anyone with a folder of flat pixel art and a scene in their head — how to turn still images into a cutscene with depth, timing and impact, and why the tool is built the way it is.

AUDIENCE

Game makers, artists, hobbyists. No coding needed.

UPDATED

September 2026

CONTENTS

What's inside

- 01** What this is for
- 02** 60-second tour
- 03** Core flows
- 04** Mental model
- 05** Build-along: an attack
- 06** Build-along: a story beat
- 07** Tips that save hours
- 08** Ideas off the beaten path
- 09** Troubleshooting
- 10** Glossary
- 11** Where to go next

START HERE

What this is for

You have flat pixel-art images — a sky, some hills, a tank, an explosion. This app stacks them at different distances and slides them past each other so the scene looks three-dimensional, then lets you time all of it into a short cinematic moment you can drop into a game.

That sliding-at-different-speeds trick is called **parallax**. It is the thing your eyes do when you look out of a car window: the fence posts rip past, the far hills barely move. Give each of your flat images its own speed and the brain reads depth that was never in the artwork.

It is built for a specific kind of moment. A unit attacks and the game cuts to a two-second close-up. A character says something that matters. You arrive somewhere new and the place name fades up over the landscape. Short, authored, repeatable.

This guide will not cover writing code, and you do not need any. It also will not cover making the art — it assumes you already have PNG files. Everything here happens by clicking, dragging and typing numbers.

“A cutscene is a list of shots. A shot is a stack of pictures. Everything else in the app is a way of saying *when*.”

THE WHOLE MENTAL MODEL IN TWO SENTENCES

ORIENTATION

60-second tour

Four areas. Left is what you are building, middle is what it looks like, right is what you are changing, bottom is when things happen.

1

Left — Shots and Assets

The top half lists your **shots**, in playing order, with a little chip between each one showing the transition. The bottom half holds every image you have imported. Drag PNG files anywhere on the window to add them. Do not look here for settings — this is just inventory.

2

Middle — the Stage

Your cutscene, playing. You can grab a layer directly on the canvas and drag it, and scroll the wheel to make it bigger or smaller. Underneath sit the play controls and the timeline.

3

Right — Layers and the Inspector

The top is the **layer stack** for whichever shot is selected: the front-most picture at the top, the furthest back at the bottom. Below it, tabbed panels change whatever you have selected. This is where nearly all your time goes.

4

Bottom — the Timeline

A bar showing every shot end-to-end, with a pink playhead. Under it, a thin strip showing **keyframes** — the moments where you have said “be this value at this time.” The payoff: press `space` and watch.

A cutscene



Shots play left to right. Inside one, layers are listed front-first and drawn back-first — exactly like sheets of painted glass stood in a row.

DO THIS • WHY IT WORKS

Core flows

Six jobs that cover most of what you will ever do. Each one shows the clicks, then explains what the app is actually doing so you can predict it next time.

1. Build a scene with depth

The first thing you will do with a fresh shot: turn a pile of flat pictures into somewhere that feels like a place.

WHAT YOU DO

1. Drag your PNG files onto the window.
2. Click **+ Image** once per picture, furthest-back first.
3. Click an asset thumbnail to load it into the selected layer.
4. Set **anchor** to *bottom edge* for anything standing on the ground.
5. In **Depth & motion**, set **parallax**: about **1.6** for the nearest, **1.0** for the subject, **0.3** for hills, **0.05** for sky.
6. In the **Cam** tab, set **pan x** to about 30.

WHY IT WORKS

pan x moves the whole scene sideways over the length of the shot. Each layer multiplies that movement by its own

parallax number. A layer at 1.6 travels further than one at 0.3, so the near things rush and the far things drift.

The numbers are not magic — they are just ratios. What matters is that they are *different*, and that they get smaller as you go back.

WHY THIS MATTERS

This is the difference between a slideshow and a shot. Two minutes of parallax numbers does more for a scene than an hour of extra art.

WATCH FOR

Nothing moves? You probably have no camera pan. Parallax on its own does nothing — it needs something to multiply. Either set **pan x**, or

give layers a `speed x`, which drifts them independently of the camera.

2. Make something move exactly when you want

A **keyframe** is a note that says “at 1.2 seconds, this value should be 96.” Put down two and the app fills in everything between them.

WHAT YOU DO

1. Drag the playhead to where the movement should start.
2. Set the value you want — say drag the layer to the left edge.
3. Click the small  beside that slider. It turns into a filled diamond.
4. Move the playhead to where the movement should end.
5. Change the value again. A key is written automatically.
6. Press `space`.

WHY IT WORKS

Once a value has a track, the app stops treating it as a fixed number and starts asking “what should this be right now?” Changing it later edits the key at the playhead rather than overwriting everything.

That is why you set the playhead *before* you change the value. The playhead decides which moment you are editing.

WHY THIS MATTERS

Position, size, opacity, colour, blur, even the size of a dialogue box — anything with a diamond can be animated the same way.

WATCH FOR

Movement that feels robotic is usually linear timing. Double-click a keyframe diamond for the curve editor and pick *ease out* — fast then settling — which is how almost everything in the physical world moves.

3. Play a sprite sheet

WHY IT WORKS

WHAT YOU DO

1. Click **+ Anim**, then click your sprite sheet in the assets list.
2. Press **Auto-detect grid**.
3. Tick **show grid** to see the slice box on the canvas.
4. If it is wrong, type **frame w** and **frame h** by hand.
5. Set **fps**, and untick **Loop** for one-shot effects like explosions.

A sprite sheet is one image holding many frames in a grid. Auto-detect looks for the empty gaps between them. It works about three times in four; it fails when the art touches or crosses the gaps — a diagonal beam, or an explosion that spills past its cell.

Animated GIFs work too. Drop one in and the app decodes every frame and lays them out as a sheet for you, frame size and speed already filled in.

WATCH FOR

Some AI generators paste your *input* picture into the first cell, which shows up as a grey box for one frame. Set **first frame** to 1 and it is skipped — no need to edit the file.

4. Make a hit feel like a hit

Impact is not one setting. It is four small ones landing together, and the order they fire in is what sells it.

WHAT YOU DO

1. Set the shot's transition to *flash*, about 0.14s.
2. **FX** tab → **Impact flash** strength around 0.9.
3. **Cam** tab → **shake** about 5, with **decay** ticked.
4. **Shot** tab → **+ Freeze at playhead** at the moment of contact.
5. On the explosion layer only, set **ignore hit stop** to 1.

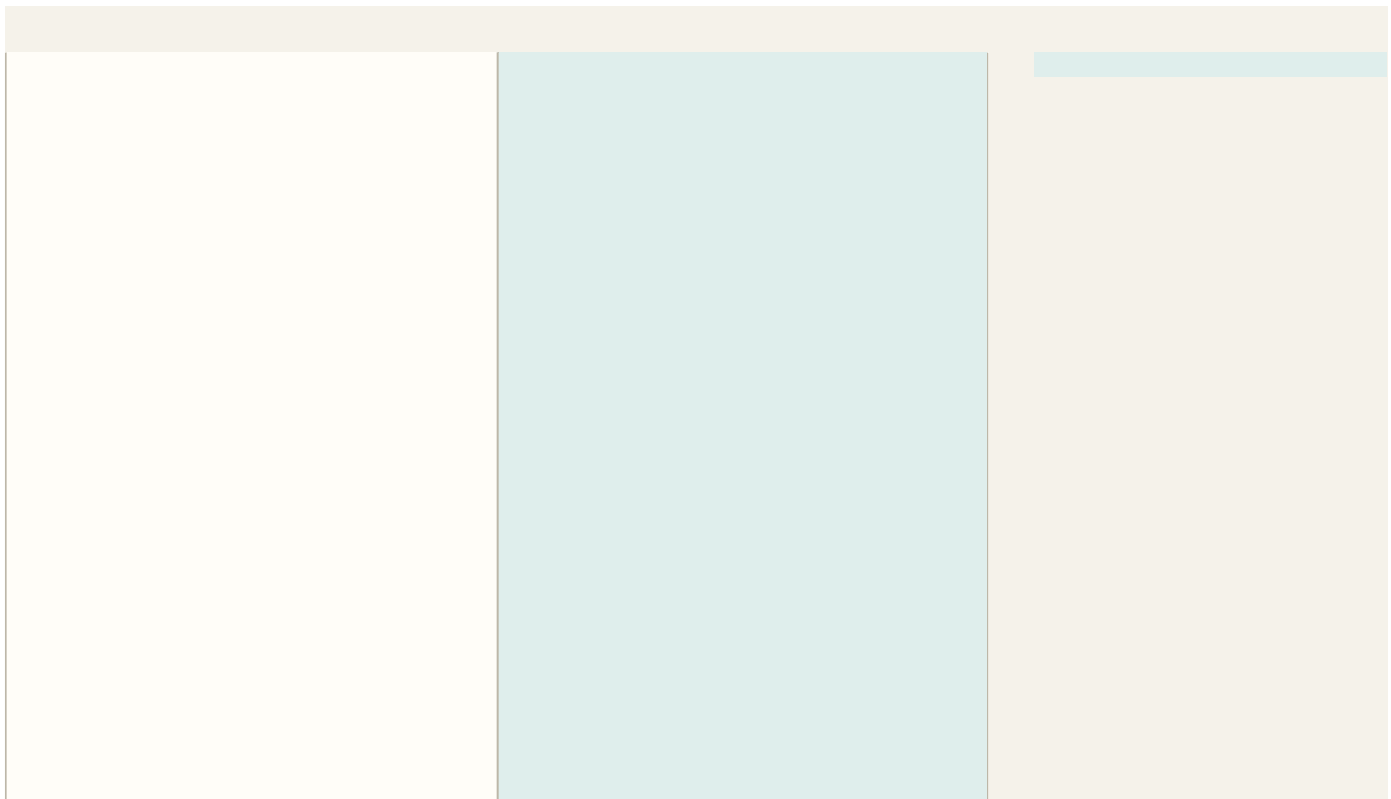
WHY IT WORKS

A freeze stops the shot for a fraction of a second. If *everything* stops, it reads as a dropped frame — a glitch. Setting the explosion to ignore the freeze means the blast keeps blooming while the world holds still, and the contrast between them is what your eye reads as weight.

Three or four frames is plenty. Longer and it stops feeling like force and starts feeling broken.

WHY THIS MATTERS

This one trick is most of the difference between an attack that looks fine and one players re-watch.



5. Write a conversation

WHAT YOU DO

1. **+ Panel**, then load a box image and set the four **inset** values to its border thickness.
2. Set **box width** and **box height** to the size you want.
3. **+ Text**, type your line, tick **wrap**, set **wrap width** a little narrower than the box.
4. Set **reveal chars/s** to about 26.
5. Use **{SPEAKER}** in a second text layer, and define it in the **Proj** tab.

WHY IT WORKS

The insets mark the border of your box art. Corners are drawn at their real size, edges are stretched, the middle fills. That is why one small image can be a box of any size without the corners going rubbery.

Anything in **{CURLY_BRACES}** is a blank your game fills in at playback. So a single dialogue shot can serve every conversation in the game — you author how a conversation *looks*, once.

6. Get it out of the app

WHY IT WORKS

WHAT YOU DO

1. **Export** tab.
2. **Save project** to keep editing later.
3. **Record WebM** for a video file.
4. **Export PNG sequence** for individual frames.
5. **Export cutscene data** for your game to play live.

The first four bake pixels. The last one exports a *recipe* — a small text file listing every layer, position and timing, with no image data at all.

Baked output works anywhere but is fixed forever. The recipe is tiny, renders at any resolution, and lets you swap which unit appears without re-exporting. If you are putting this in a game, you want the recipe.

HOW TO THINK ABOUT IT

The mental model

Five ideas. Hold these and the rest of the app stops needing to be memorised, because you can work out where things must be.

There is only one kind of shot

There is no separate “animation mode.” An explosion cut is just a shot whose layer happens to be a sprite sheet. That is why you can put an explosion on top of a live scene, or chain three animations together — the app never had two modes to switch between.

Time is the only input

The picture you see depends on nothing but the current time. Nothing accumulates. This is why scrubbing backwards works perfectly, why an exported video matches the preview exactly, and why the same file plays identically in your game.

Front to back, top to bottom

The layer list is ordered like a stack of cards seen from above: the top of the list is nearest your eye. The app draws from the bottom up, so later entries end up behind.

Layers say what, the timeline says when

The right panel sets how something looks. The bottom strip sets when it changes. Any time you think “but I want it to do that *later*,” the answer is a keyframe or a marker, not a different setting.

Slots keep you from doing the same job fifty times

Instead of pointing a layer at *tank_blue.png*, point it at the role `{ATTACKER}`. Your game supplies whichever unit is actually attacking. One cutscene, entire roster.

“If you are about to build the same shot twice, stop. There is a slot, a variable, or a library template that means you only build it once.”

THE RULE THAT SAVES THE MOST TIME

BUILD-ALONG ONE

A tank attacks

Four shots, about nine seconds. A unit advances, charges, fires, and the target takes the hit. This is the workhorse cutscene — if your game has combat, you will build variations of this more than anything else.

YOU WILL NEED	Four or five background layers, a unit sprite (or sheet), and an explosion sheet.
TIME	About twenty minutes the first time, five once you know it.
YOU WILL USE	Parallax, keyframes, camera pan and zoom, hit stop, masks, groups, events.

Shot 1 — Advance (3.2s)

Build the scene as in flow 1. Add your unit as a layer above the ground and below the foreground, `anchor` bottom, `parallax` 1.0.

Now drive it in. Put the playhead at 0, drag the unit off the left edge, click its  for `offset x`. Move the playhead to 2.6s, drag the unit to just left of centre. Double-click the first diamond and choose *ease out* so it decelerates as it arrives.

In `Cam`, set `pan x` 28 and `zoom end` 1.07 — a slow push in. In `FX`, `vignette` 0.44, `film grain` 0.08, `letterbox` 0.15.

Shot 2 — Charge (2.0s)

Add a shot, choose *Animation over scene* — it copies the shot you are on so the background stays put. Swap the unit to its charging animation if you have one, and add an energy sheet on top with `blend` set to *add*, which makes bright pixels glow instead of sitting flatly on top.

Build pressure: `zoom start` 1.07 → `zoom end` 1.18, easing *in* so it accelerates. Add `shake` 1.2 and *untick decay* so the rumble holds rather than fading.

Then key the colour. In `FX`, click the  on `wash amt` at 0s with a value near zero, move to the end of the shot, raise it to about 0.3. The whole frame now bleeds toward the charge colour as the shot builds.

Shot 3 — Impact (2.0s)

The important one. Transition *flash* at 0.14s. Add the explosion sheet centred on the target, `loop` off, `blend` add.

Add the freeze at 0.12s for 0.12s, then set `ignore hit stop` to 1 on the explosion layer only.

For the white hit flash: add a `+ Solid` layer set to white, place it directly above the target in the stack, and set its `mask with` to the target layer. It is now clipped to the target's silhouette. Key its `opacity`: 0 at 0s, 0.95 at 0.07s, back to 0 by 0.4s.

Camera: `zoom start` 1.18 → `zoom end` 1.0, `shake` 5.5 with decay on. FX: impact flash 0.9, `chromatic` 2.3 for that split-colour fringe.

Finally, `Shot` → **+ Event at playhead** at 0.12s, name it `impact`. Your game listens for that and spawns a scorch mark at exactly the right instant.

Shot 4 — Aftermath (2.4s)

Transition *pixel dissolve* at 0.5s. Same scene, target darkened — `brightness` 0.45, `saturation` 0.4, a dark `tint` at 0.5.

Add a text layer reading `-{DAMAGE}`. Key its opacity up fast, hold, fade out; key its `offset y` drifting upward across the whole shot. Set `DAMAGE` in the `Proj` tab so you can see it, but your game will supply the real number.

THE SHORTCUT

If you have set up a unit in the `Units` tab with its actions, press **Generate attack scene** and the app builds all four shots with timing, markers and a shockwave already in place. Then edit. It is a strong first draft, not a finished scene.

BUILD-ALONG TWO

Arriving somewhere, and a conversation

A location reveal followed by two lines of dialogue. Almost no motion — this one is about restraint, and it is where a game starts to feel like it was made by someone who cared.

YOU WILL NEED	A landscape in layers, a box image for dialogue, a character sprite.
TIME	Fifteen minutes.
YOU WILL USE	Groups, 9-slice panels, text wrap and reveal, variables, crossfades.

Shot 1 — Approach (3.6s)

Build the landscape with parallax, then give every background layer a gentle `speed x` of -22 down to -1.2 , near to far. That is drift without a camera move — the world is alive but you are not being pushed anywhere.

Open with *fade* over a full second. Long fades read as “time has passed”; short ones read as “meanwhile.”

Now group them. `Shot` → **+ Add group**, call it `world`, and set every background layer’s `group` to it. Key the group’s `offset y` from 10 down to 0 across the shot. The entire landscape settles as one, on one track instead of six.

Shot 2 — The name card (4.0s)

Halve all the drift speeds so the world nearly stops. Add a `+ Solid` dark layer above the landscape and key its `opacity` to about 0.42 — a scrim, so white text has something to sit on.

Add two text layers: `{PLACE}` large with `tracking` 3, and `{REGION}` small beneath it. Define both in `Proj`.

Stagger them. The place name fades in over the first second; the region waits until 1.8s.

Both hold, both fade before the shot ends. Give the place name a slow upward `offset y` drift of about 16 pixels across four seconds — barely perceptible, and it is what stops the card feeling like a static screenshot.

Shot 3 — The exchange (4.6s per line)

Transition *crossfade* at 0.5s. Add your panel, insets matching its border, box about 84% of frame width, anchored to the bottom.

Key the panel's `box height` from 8 to its full height over 0.28s. The box opens rather than appearing.

Add the line as a text layer with `wrap` on and `reveal chars/s` 26, with a `reveal delay` of 0.4s so the box finishes opening before the text starts. Add the speaker name above it in the accent colour.

For the second line, duplicate the shot and change the text and speaker. Nudge the zoom slightly further in on each line — 1.0, then 1.04, then 1.1 — and the conversation tightens as it gets more serious, without a single camera move the player will consciously notice.

WATCH FOR

Reveal speed is worth tuning by ear, not by number. Play it back and read along. If you finish reading before the text does, it is too slow — and slow text is the single most common complaint about dialogue in games.

EARNED THE HARD WAY

Tips that save hours

Loop the moment, not the movie

Press  and  to mark a loop region around the half-second you are tuning. You will iterate ten times faster than replaying from the start.

Markers make things land together

Add a marker at the moment of impact, and every keyframe you drag will snap to it. Five layers hitting the same instant exactly, not nearly.

Onion skin for arcs

 ghosts the selected layer at its neighbouring keyframes — magenta behind, cyan ahead. You can see the path of a motion instead of imagining it.

Grain hides your seams

A little film grain, 0.08 to 0.14, ties mismatched art together and softens banding in gradients. It is the cheapest polish in the app.

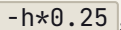
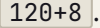
Copy layers between shots

The  and  buttons move a layer across shots and bring its image with it. Faster than rebuilding a background five times.

Shorter than you think

Two seconds feels long in a game that plays a cutscene every turn. Build it, then cut a third out.

Type maths into number boxes

Any number field accepts , , . Centring things stops being guesswork.

Solo when it gets crowded

The  button hides everything except the selected layer. The fastest way to find out which layer is actually causing the problem.

Letterbox is free cinema

Black bars at 0.15 instantly say “this is a cutscene, pay attention.” Keep it consistent across every shot in a scene.

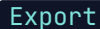
Save the shot, not just the project

 → **Save shot** stores it in a library with its images. Build your dialogue shot once and insert it into every future project.

Ease out is almost always right

Real things start fast and settle. If a movement feels mechanical, that is usually all that is wrong with it.

Check the live sync

 → **Choose sync folder** writes the data file on every change. Point your game at it and tune the cutscene against real gameplay.

OFF THE BEATEN PATH

Things the app can do that it was not really built for

Every one of these uses only what is already in the tool. They exist because the pieces compose — a mask does not know or care that you are using it for a map wipe instead of a hit flash.

Animated menu backgrounds

Build a slow parallax loop, export a WebM, play it behind your title screen. No camera move, gentle drift, heavy grain.

Animated map transitions

Put your map image on one layer and a shape on another, set the map to *mask with* the shape, and key the shape growing. Now your map wipes open in whatever shape you like.

Heat haze over a desert

A distortion layer in *shimmer* mode, low amount, covering just the bottom third. It only bends what is behind it, so your foreground stays crisp.

Sprite sheets from cutscenes

Export a sprite sheet instead of a video, and your whole cutscene becomes one image your engine can flip through — useful on very constrained platforms.

Loading screens that are not static

A one-second parallax loop with tiling on, exported as a PNG sequence, loops forever without a seam.

Weather as a layer

A tiling rain sheet with high `speed y`, low opacity, motion blur at 1. Drop it into any scene. Do it twice at different speeds and parallax values and you have depth in the rain too.

Damaged-signal cutscenes

A distortion layer in *warp* mode with its amount keyframed to spike briefly, plus scanlines and chromatic aberration. A transmission breaking up, without a single new asset.

Comic panels

Several 9-slice panels at different sizes, each keyed to appear in sequence, with art masked inside them. Motion comics with no new tools.

A character who is never drawn

Give a solid colour layer a mask of your character sprite and never show the sprite itself. Now they are a silhouette — for a reveal later, or a character you never see properly.

One scene, two endings

Use the **Sequence** panel to branch on a variable. The same file can play the victory or the retreat depending on what your game passes in.

Recolour instead of redraw

Build a palette from a sprite, recolour the swatches, and the same art serves every faction. Half the art budget, gone.

Fake 3D rotation

Key a layer's horizontal scale from 1 to 0 and swap to a different sprite at the moment it is edge-on. A card flip, a door, a unit turning.

Nested groups for a living city

Put the whole skyline in one group, and the neon signs in a group inside it. Move the outer group and everything drifts; move the inner one and only the signs sway.

Cut on the beat

Import your music cue and its waveform draws behind the timeline. Put your shot boundaries on the loud parts and the whole scene feels scored rather than accompanied.

“A mask does not know it is a hit flash. A distortion layer does not know it is heat. Once you see the pieces as pieces, the list of things you can build stops being a list.”

WHY COMPOSITION BEATS FEATURES

WHEN IT GOES WRONG

Troubleshooting

SYMPTOM	CAUSE	FIX
Nothing moves when I play	Parallax with no camera pan to multiply	Set <code>pan x</code> in the Cam tab, or give layers a <code>speed x</code>
My sprite sheet plays garbage	Frame size does not divide the sheet evenly	Tick <code>show grid</code> and adjust <code>frame w</code> / <code>frame h</code> until the box fits one frame
A grey box flashes for one frame	The generator pasted your input image into cell one	Set <code>first frame</code> to 1
Last row of frames is missing	Frame height does not divide evenly, so the row is cut	Drop the frame height by one or two pixels
Unit jumps when its animation changes	The two actions have different frame sizes	Units tab → Pivot from artwork on every action
Unit floats above the ground	Pivot is on the padded frame edge, not the art	Same fix — pivot from artwork finds the real feet
Character sheet has a magenta background	Exported without transparency	Hover the asset, click  , pick the magenta, apply
Background removal leaves a halo	The image is a JPG; its edges were blended away	Re-export as PNG — JPGs cannot hold transparency at all
Keyframe does not go where I want	The playhead was somewhere else when you changed the value	Always move the playhead first, then change the value
The hit freeze looks like a glitch	Every layer is frozen, including the explosion	Set <code>ignore hit stop</code> to 1 on the effect layer
Layers shimmer while scrolling	The art is upscaled, so its pixels are not whole pixels	Downscale the source to its true pixel size before importing

SYMPTOM	CAUSE	FIX
Dialogue box corners look stretched	Insets do not match the border thickness	Set each inset to the real border width in pixels
Frame export fails	Very long cutscene at a high upscale	Lower the upscale, or export as video instead
I closed the app and lost work	You had not saved a project file	Reopen — it offers to restore. Then save properly

PLAIN LANGUAGE

Glossary

Layer

One picture in the stack. A sky, a hill, a tank, a line of text.

Shot

One continuous view. A cutscene is several shots in a row.

Parallax

How far a layer moves compared to the camera. Bigger means nearer.

Anchor

Which edge a layer sticks to. Bottom keeps things on the ground.

Keyframe

A note saying “be this value at this time.” Two make a movement.

Easing

Whether a movement starts fast, ends fast, or is mechanical throughout.

Sprite sheet

One image holding many animation frames in a grid.

Hit stop

A deliberate freeze of a fraction of a second, to sell impact.

Mask

Using one layer’s shape as a cookie cutter for another.

Group

Several layers sharing one set of move / scale / fade controls.

9-slice

A box image cut into nine parts so it resizes without warping the corners.

Slot

A named blank — `{ATTACKER}` — that your game fills in at playback.

Unit

A list of one character's animations: idle, move, fire, hit.

Pivot

The point on a sprite that counts as its feet.

Event

A timestamp your game listens for. Never changes the picture.

Variable

A blank in a line of text, written `{LIKE_THIS}`.

Marker

A named moment. Keyframes snap to it so things land together.

Distortion layer

A layer that bends what is behind it: heat, shockwaves, bad signal.

FINALLY

Where to go next

Do not read more. Open the app, load one of the example projects, and break something.

1

Change one number at a time

Open an example, pick a parallax value, and move it a long way. Watch what happens. Undo. That is the fastest way to build intuition for the whole app.

2

Rebuild build-along one with your own art

Same four shots, your pictures. You will hit two or three snags, and the troubleshooting table almost certainly covers them.

3

Then make it shorter

Cut a third of the running time out. Almost every first cutscene is too long, and tightening is where it starts to feel professional.

Everything in this app runs on your machine, offline, with nothing sent anywhere. Your project file holds your art, your timings and your text — it is yours, and it will still open in five years.